

CAPSTONE PROJECT

Prediction of International Cricket Match Outcomes Using Machine Learning

August 2026

To
Great Learning - DSBA

Submitted by
Vasandhan Sakthivel

Table of Contents

Executive Summary 3

1. Introduction 4

2. Exploratory Data Analysis and Business Implications 5

2.1 Dataset Overview 5

2.2 Non-Visual Data Understanding 5

2.3 Visual Exploratory Data Analysis 6

2.4 Summary of Key Findings 9

3. Data Cleaning and Pre-processing 11

3.1 Missing Value Treatment 11

3.2 Outlier Assessment and Treatment 11

3.3 Variable Transformation 12

3.4 Variable Selection 13

4. Model Building 15

4.1 Model Selection Strategy 15

4.2 Model Development Process 15

4.3 Summary 16

5. Model Validation 17

6.Final Interpretation/Recommendations 20

7. Appendix 22

E X E C U T I V E S U M M A R Y

The Board of Control for Cricket in India (BCCI) continually seeks to improve team performance through informed decision-making. Predicting match outcomes using historical cricket data provides an opportunity to support team selection, tactical planning, and performance analysis using objective, data-driven insights. This study was undertaken to develop a predictive model capable of estimating the outcome of international cricket matches involving the Indian cricket team.

Historical match data was analysed to identify the factors influencing match outcomes and to develop a reliable machine learning solution. The analysis included exploratory data analysis, data preprocessing, model development, and comparative evaluation of multiple classification algorithms to identify the most suitable predictive model.

The findings demonstrate that machine learning can successfully identify meaningful patterns associated with match outcomes. Among the evaluated machine learning models, one ensemble learning algorithm demonstrated the strongest predictive performance and was selected as the final decision-support model.

The analysis highlights the importance of factors such as team composition, batting and bowling performance, opposition characteristics, and bowling discipline in influencing match results. These insights provide valuable decision support for coaches, selectors, and analysts when evaluating team combinations and preparing match strategies.

Overall, this study demonstrates that predictive analytics can complement traditional cricket expertise by providing objective evidence to support strategic decision-making. The developed model offers a practical foundation for integrating data-driven insights into future performance analysis and match preparation within the BCCI.

1. INTRODUCTION

Cricket has evolved beyond traditional decision-making, with teams increasingly relying on data analytics to improve performance and gain a competitive advantage. Historical match data provides valuable information about team composition, player performance, batting and bowling statistics, and match conditions, enabling organizations to identify patterns that influence match outcomes. The growing availability of such data has made machine learning an effective tool for supporting strategic decisions in modern cricket.

The Board of Control for Cricket in India (BCCI) aims to leverage historical match data to predict the outcome of international cricket matches. Match results are influenced by numerous factors, including team composition, opposition strength, batting and bowling performance, venue characteristics, and match conditions. Analysing these variables using machine learning techniques can help identify the key factors associated with winning or losing a match and develop predictive models capable of estimating future match outcomes.

Developing an accurate prediction model can assist the BCCI in making more informed decisions regarding team selection, tactical planning, and opponent analysis. By transforming historical data into actionable insights, predictive analytics can support evidence-based decision-making, improve match preparation, and enhance the overall effectiveness of cricket management strategies.

2. EXPLORATORY DATA ANALYSIS AND BUSINESS IMPLICATIONS

2.1 DATASET OVERVIEW

The dataset used for this study comprises 2,930 historical international cricket matches involving the Indian cricket team. It contains information describing various aspects of each match, including team composition, batting performance, bowling performance, match conditions, and the final match result. The objective of analysing this dataset is to identify the factors that influence match outcomes and to develop a predictive model capable of estimating whether India is likely to win or lose a match.

Before building predictive models, it is essential to understand the structure and characteristics of the data. Exploratory Data Analysis (EDA) was performed to examine the distribution of variables, identify relationships among features, detect data quality issues, and generate business insights that support evidence-based decision-making.

Table 2.1 Dataset Summary

Attribute	Description
Total Records	2,930 matches
Total Variables	23
Numerical Variables	15
Categorical Variables	8
Target Variable	Result (Win/Loss)

2.2 NON-VISUAL DATA UNDERSTANDING

A preliminary examination of the dataset was conducted using descriptive statistics to understand the characteristics of the numerical variables. The analysis revealed that the average team age was approximately 29 years, indicating that the Indian team generally consisted of experienced players. However, unrealistic age values were identified, highlighting the need for data cleaning before model development.

Attendance figures varied considerably across matches, reflecting differences in venue capacity, tournament importance, and spectator interest. Similarly, batting and bowling performance indicators exhibited substantial variability, suggesting that these variables are likely to contribute meaningful information when predicting match outcomes.

Overall, the descriptive analysis indicates that the dataset contains sufficient variation across both player performance and match conditions to support predictive modelling while also identifying areas requiring preprocessing.

Table 2.2 Key Descriptive Observations

Variable	Observation	Business Insight
Average Team Age	Mean ≈ 29 years	Indian squads generally consist of experienced players.
Audience	Highly variable	Attendance differs significantly across venues and tournaments.
Batting Performance	Wide range of scores	Individual batting performances vary considerably between matches.
Bowling Performance	Noticeable variation	Bowling effectiveness differs across matches and may influence outcomes.

2.3 VISUAL EXPLORATORY DATA ANALYSIS

Visual analysis was performed to better understand the distribution of variables and the relationships that may influence match outcomes.

2.3.1 Univariate Analysis

Match Result

The distribution of the target variable indicates that the dataset contains substantially more winning matches than losing matches. This imbalance suggests that relying solely on prediction accuracy could produce misleading conclusions during model evaluation. Consequently, additional performance metrics such as Precision, Recall, F1-score, and ROC-AUC should be considered during model assessment.

Recognising the class imbalance at an early stage ensures that the predictive model is evaluated fairly and is capable of identifying both winning and losing scenarios accurately. This improves the reliability of predictions used to support strategic cricket decisions.

Note**: Supporting Distribution of Match Results are provided in **Appendix B Figure B.1

Average Team Age

The distribution of average team age is concentrated around the late twenties, indicating that India has generally fielded experienced teams. However, a small number of unrealistic observations were identified, suggesting the presence of data quality issues that require preprocessing before model development.

Maintaining accurate player information is essential for building reliable predictive models. Correcting anomalous values improves data quality and ensures that team selection insights are based on realistic player characteristics.

Note**: Supporting Average Team Age Results are provided in **Appendix B Figure B2

Audience Attendance

Audience attendance exhibits a positively skewed distribution with several extreme observations. The variation indicates that crowd sizes differ considerably depending on venue, tournament significance, and match popularity.

Understanding attendance patterns can support planning for high demand matches and provides additional contextual information that may indirectly influence player performance and match environments.

Note**: Supporting Audience attendance Results are provided in **Appendix B Figure B3

2.3.2 Bivariate Analysis

Bivariate analysis was conducted to examine the relationship between two variables and to identify factors associated with match outcomes. Understanding these relationships helps determine which variables have the greatest influence on winning or losing a match and provides valuable insights for strategic decision-making.

Match Format and Match Result

The analysis indicates that India's performance varies across different match formats. Although One Day Internationals (ODIs) account for the largest proportion of matches in the dataset, the percentage analysis demonstrates that the likelihood of winning is not identical across ODI, Test, and T20 formats. These differences suggest that match format influences team performance and should be considered when evaluating historical success.

Different cricket formats require distinct playing strategies, squad compositions, and tactical approaches. Historical performance across formats can support format-specific planning, enabling coaching staff to optimise team selection and match preparation for each type of international fixture.

Note: *Supporting countplot and percentage crosstab are provided in* ***Appendix B (Figure B4 and Table B4)****.*

Opponent and Match Result

Historical match results indicate that India's winning performance differs across opponents. While the number of matches played against each team varies, percentage-based analysis reveals that certain opponents have consistently posed greater challenges than others. This suggests that the strength and characteristics of the opposition influence match outcomes.

Understanding historical performance against different opponents enables the coaching staff to identify challenging match-ups and develop opponent-specific strategies. These insights can assist in tactical planning, player selection, and performance analysis before future international tournaments.

Note: *Supporting countplot and percentage crosstab are provided in* ***Appendix B (Figure B5 and Table B5)****.*

First Selection and Match Result

The relationship between the first selection after winning the toss and the final match result was examined to determine whether choosing to bat or bowl first influences performance. The analysis suggests that match outcomes vary depending on the initial decision, although the magnitude of this effect differs across matches.

Winning the toss provides an important strategic opportunity. Historical trends relating batting-first and bowling-first decisions to match outcomes can support captains in making more informed decisions based on pitch conditions, weather, and opposition characteristics.

Note: *Supporting countplot and percentage crosstab are provided in* ***Appendix B (Figure B6 and Table B6)****.*

Extra Deliveries Bowled and Maximum Runs Conceded

The relationship between the number of extra deliveries bowled and the maximum runs conceded in a single over was analysed using a scatter plot. The analysis indicates a positive relationship between these variables, suggesting that matches with a higher number of extra deliveries are generally associated with higher scoring overs. This finding highlights the impact of bowling discipline on overall match performance.

Reducing unnecessary extra deliveries can help limit opposition scoring opportunities and improve bowling efficiency. Monitoring this performance indicator enables coaches to identify areas for improvement and implement targeted training interventions aimed at strengthening bowling discipline.

Note: *Supporting scatter plot is provided in* ***Appendix B (Figure B7)*****.**

2.3.3 Multivariate Analysis

Multivariate analysis was conducted to examine the combined relationships among multiple numerical variables and to identify potential dependencies that could influence predictive model performance. Understanding these relationships helps determine whether variables contribute unique information or introduce redundancy into the dataset.

Correlation Analysis

The correlation analysis indicates that most numerical variables exhibit weak to moderate relationships with one another. Although certain bowling-related variables show positive associations, no pair of variables demonstrates an exceptionally high correlation that would indicate severe multicollinearity. This suggests that the variables capture different aspects of team performance and collectively provide meaningful information for predicting match outcomes.

The absence of severe multicollinearity improves the reliability and interpretability of predictive models. Since the variables contribute complementary information rather than duplicating one another, the models can better identify the factors influencing match outcomes. This enables the BCCI to make data-driven decisions using insights derived from multiple dimensions of team and match performance.

Note: *The complete correlation heatmap is provided in* ***Appendix B (Figure B8)****.*

2.4 SUMMARY OF KEY FINDINGS

The exploratory data analysis provided several important insights that informed the subsequent stages of model development:

- The dataset contains an imbalance between winning and losing matches, requiring careful selection of evaluation metrics during model assessment.
- Data quality issues, including unrealistic player age values and outliers, were identified and addressed during preprocessing.

- Historical performance varies across match formats, opponents, and first-selection decisions, indicating that contextual factors influence match outcomes.
- Bowling discipline, particularly the control of extra deliveries, emerged as an important performance indicator associated with match success.
- Correlation analysis confirmed that the predictor variables provide complementary information without severe multicollinearity, supporting the development of robust machine learning models.

Overall, the exploratory analysis demonstrated that the dataset contains meaningful patterns capable of supporting predictive modelling while providing actionable insights for strategic cricket decision-making.

3. DATA CLEANING AND PRE-PROCESSING

Before developing predictive models, the dataset was pre-processed to improve data quality and ensure that the variables were suitable for machine learning algorithms. The preprocessing stage focused on handling missing values, assessing outliers, transforming variables where necessary, and selecting the most relevant features for model development. These steps improve model reliability, reduce bias, and enhance predictive performance.

3.1 MISSING VALUE TREATMENT

A review of the dataset identified missing values in both numerical and categorical variables. Rather than removing observations with incomplete information, missing values were imputed to preserve the available data and minimise information loss.

Missing values in numerical variables were replaced using the median, while missing values in categorical variables were replaced using the most frequently occurring category (mode).

The median was selected because it is less sensitive to extreme observations than the mean, making it more suitable for variables exhibiting skewed distributions. Similarly, mode imputation preserves the most representative category without introducing unrealistic values into categorical variables.

Retaining observations through appropriate imputation increases the amount of information available for model training while reducing the risk of biased predictions resulting from unnecessary data removal.

Note: *The Python implementation for missing value treatment is provided in* ***Appendix C2****.*

3.2 OUTLIER ASSESSMENT AND TREATMENT

Exploratory analysis identified several variables containing extreme observations, particularly those related to team age, audience attendance, and match performance statistics. These observations were examined to determine whether they represented genuine match events or data quality issues.

Unrealistic observations, such as implausible team age values, were treated as data quality concerns during preprocessing. However, performance-related extreme values were retained because they represent valid cricket events that may contain valuable predictive information. Removing genuine match performances could reduce the model's ability to learn important patterns associated with exceptional victories or defeats.

Appropriate outlier treatment improves data quality while ensuring that legitimate high-impact performances remain available for model training. This enables the predictive models to capture both typical and exceptional match scenarios.

Note: *The Python implementation for missing value treatment is provided in* ***Appendix_C3****.*

3.3 VARIABLE TRANSFORMATION

To ensure compatibility with the machine learning algorithms, several preprocessing transformations were applied before model development.

Integer-based performance variables were converted to appropriate integer data types to maintain consistency within the dataset. Additionally, categorical variables were transformed into numerical representations using One-Hot Encoding, allowing the classification models to interpret categorical information without assuming an ordinal relationship between categories. The first category was excluded during encoding to reduce redundancy and minimise multicollinearity among the generated dummy variables.

For the K-Means clustering analysis, numerical variables were standardised prior to clustering to ensure that all variables contributed equally to the distance calculations. Since K-Means relies on Euclidean distance, feature scaling prevented variables with larger numerical ranges from dominating the cluster formation process. Following the clustering process, a new variable named Cluster was created to assign each observation to its corresponding cluster, enabling segmentation and interpretation of groups with similar match characteristics.

Transforming categorical variables into numerical features and standardising numerical variables ensured that both the classification models and the clustering algorithm could effectively utilise the available information, thereby improving the reliability and interpretability of the analytical results.

Note: *Variable transformation procedures are provided in* ***Appendix C4****.*

3.4 VARIABLE SELECTION

Not all variables were considered suitable for predictive modelling. Variables that did not contribute meaningful predictive information or could potentially introduce bias into the model were excluded.

Variable	Reason for Removal
Game_number	Unique identifier that does not contribute to predicting match outcomes.
Audience_number	Excluded from model development because it is unlikely to directly influence match outcomes and may introduce unnecessary variability.
Wicket_keeper_in_team	Removed due to limited variability, reducing its potential contribution to model performance.

Following variable selection, the target variable (**Result**) was separated from the predictor variables before model training.

Variables Removed for K-means Clustering

Variable	Reason for Removal
Result	Removed because K-Means is an unsupervised learning algorithm and therefore does not use a target variable.
Game_number	Excluded because it is a unique identifier and does not contribute to meaningful cluster formation.

Variable Added

Variable	Reason for Addition
Cluster	Created after applying the K-Means clustering algorithm to assign each observation to a cluster, facilitating segmentation and interpretation of similar match characteristics.

Removing non-informative variables simplified the analytical models, reduced unnecessary complexity, and enabled both the classification models and the clustering algorithm to focus on variables that provided meaningful information.

Note: *Feature selection code is provided in* ***Appendix C4****.*

Summary of Data Pre-processing

The preprocessing stage significantly improved the quality and suitability of the dataset for both predictive modelling and clustering analysis. Missing values were appropriately imputed, data quality issues were addressed, categorical variables were transformed into machine-readable formats, numerical variables were standardized where required, and non-informative variables were removed. In addition, a Cluster variable was generated following the K-Means clustering analysis to support segmentation of observations into groups with similar characteristics. These preprocessing decisions ensured that the dataset was consistent, reliable, and well-prepared for both supervised and unsupervised machine learning techniques.

4. MODEL BUILDING

4.1 MODEL SELECTION STRATEGY

Following data preprocessing, multiple supervised machine learning algorithms were developed to predict the outcome of international cricket matches. Since the target variable (Result) consists of two classes (Win and Loss), classification algorithms were selected for model development.

Rather than relying on a single algorithm, four different classification models representing distinct modelling approaches were evaluated. This approach enabled a comprehensive comparison of model performance and ensured that the final predictive model was selected based on objective evaluation rather than algorithm preference.

The selected models include:

- **Logistic Regression** – selected as the baseline classification model because of its simplicity, interpretability, and suitability for binary classification problems.
- **Decision Tree** – selected to capture non-linear decision boundaries and complex interactions among predictor variables.
- **Random Forest** – selected as an ensemble learning technique that combines multiple decision trees to improve prediction accuracy and reduce overfitting.
- **Gradient Boosting** – selected to evaluate whether sequential boosting of weak learners could achieve superior predictive performance compared with the other models.

Evaluating models with varying levels of complexity provided a robust framework for identifying the most appropriate algorithm for predicting cricket match outcomes.

4.2 MODEL DEVELOPMENT PROCESS

The cleaned dataset was divided into training and testing subsets to evaluate the ability of each model to generalise to unseen data. A consistent train-test split was maintained across all models to ensure fair performance comparison.

Logistic Regression requires predictor variables to be on a comparable scale; therefore, numerical features were standardised prior to model training. In contrast, tree-based models such as Decision Tree, Random Forest, and Gradient Boosting were trained using the original feature values, as these algorithms are not affected by differences in variable scales.

This standardised modelling framework ensured consistency throughout the model development process and enabled meaningful comparison between algorithms.

Model Performance Improvement

This is the most important subsection because it directly answers the second part of the rubric.

Several optimisation techniques were implemented to improve predictive performance and ensure that the developed models were both accurate and reliable.

Handling class imbalance

Since the dataset contained a greater proportion of winning matches than losing matches, balanced class weights were applied where appropriate to reduce bias towards the majority class.

Hyperparameter tuning

Hyperparameter optimisation was performed using GridSearchCV to identify the optimal parameter combinations for the Decision Tree, Random Forest, and Gradient Boosting models. Parameters controlling model complexity and learning behaviour were systematically evaluated to improve generalisation while reducing overfitting.

Model validation

Multiple evaluation metrics, including Accuracy, Precision, Recall, F1-score, and ROC-AUC, were used to assess model performance. Evaluating several metrics ensured that model selection was not based solely on accuracy but also considered the model's ability to correctly identify both winning and losing matches.

These optimisation strategies improved model robustness and provided confidence in selecting the most appropriate predictive model.

4.3 SUMMARY

Four classification algorithms representing different modelling approaches were developed and optimised using a consistent training framework. Appropriate preprocessing, feature scaling, class balancing, and hyperparameter tuning techniques were applied to improve predictive performance and minimise overfitting.

The performance of these models is compared in the following section to identify the algorithm that provides the most accurate and reliable prediction of cricket match outcomes.

5. MODEL VALIDATION

Validation Strategy

Following model development, each classification model was evaluated using a separate testing dataset to assess its ability to generalise to unseen observations. Using an independent test dataset ensured that the reported performance reflected the model's predictive capability rather than its ability to memorise the training data.

As the dataset contained a higher proportion of winning matches than losing matches, model validation was not based solely on prediction accuracy. Multiple evaluation metrics were used to provide a comprehensive assessment of model performance and to ensure that the selected model could reliably classify both winning and losing outcomes.

Model Evaluation Metrics

To evaluate the predictive performance of each classification model, multiple evaluation metrics were considered.

- **Accuracy** was used to measure the overall proportion of correctly classified matches.
- **Precision** evaluated the reliability of the model's predictions.
- **Recall** measured the model's ability to correctly identify the actual match outcomes.
- **F1-score** provided a balanced assessment by considering both Precision and Recall.
- **ROC-AUC** measured the model's ability to distinguish between winning and losing matches across different classification thresholds.

Using multiple evaluation metrics ensured that model selection was based on overall predictive performance rather than relying on a single performance measure.

Comparative Performance of Models

Model	Train Accuracy	Test Accuracy	Weighted Precision	Weighted Recall	Weighted F1-score	ROC-AUC
Logistic Regression	74.88%	73.77%	84%	74%	77%	82.57%
Decision Tree	100.00%	93.04%	93%	93%	93%	87.15%
Random Forest	96.97%	91.11%	91%	91%	91%	93.06%
Gradient Boosting	100.00%	95.10%	95%	95%	95%	96.86%

Logistic Regression served as the baseline model and established an initial performance benchmark. Although it provided reasonable predictive capability, its comparatively lower accuracy indicates that a linear classification model is less effective in capturing the complex relationships within the dataset.

Decision Tree significantly improved predictive performance; however, its perfect training accuracy suggests a tendency towards overfitting. Random Forest reduced this limitation through ensemble learning, resulting in improved generalisation and stable performance across the evaluation metrics.

Among the evaluated models, Gradient Boosting achieved the highest overall performance, recording a testing accuracy of 95.10%, weighted Precision of 95%, weighted Recall of 95%, weighted F1-score of 95%, and an ROC-AUC value of 96.86%. These results demonstrate that the model provides the most accurate and balanced predictions across both outcome classes.

Selection of the Final Model

Based on the comparative evaluation, Gradient Boosting was selected as the final predictive model for this study. The model consistently outperformed the other algorithms across all evaluation metrics while maintaining strong generalisation on the testing dataset. Unlike the Decision Tree model, which exhibited signs of overfitting due to perfect training accuracy, Gradient Boosting achieved superior predictive performance while remaining reliable on unseen data.

The strong ROC-AUC value further demonstrates the model's ability to effectively distinguish between winning and losing match outcomes, making it the most suitable model for supporting data-driven decision-making within the Board of Control for Cricket in India (BCCI).

Summary

Model validation confirmed that evaluating multiple performance metrics provides a more comprehensive assessment than relying solely on prediction accuracy. The combined use of Accuracy, Precision, Recall, F1-score, and ROC-AUC enabled an objective comparison of all four classification models. Based on the evaluation results, Gradient Boosting was identified as the most accurate and reliable model for predicting international cricket match outcomes and was therefore selected as the final predictive model.

Note: *Detailed confusion matrices, classification reports, ROC curves, and hyperparameter tuning results are provided in* ***Appendix D*** *and* ***Appendix E****.*

6. FINAL INTERPRETATION / RECOMMENDATIONS

The objective of this study was to develop a machine learning model capable of predicting the outcome of international cricket matches using historical match data. The exploratory analysis identified several factors associated with match performance, including team composition, batting and bowling performance, opposition characteristics, and match conditions.

Following data preprocessing and model development, four classification algorithms were evaluated. Comparative analysis demonstrated that Gradient Boosting consistently achieved the strongest predictive performance, recording the highest testing accuracy and superior results across Precision, Recall, F1-score, and ROC-AUC. These findings indicate that machine learning can effectively identify patterns within historical cricket data and support evidence-based decision-making.

Strategic Recommendations for BCCI

Area	Recommendation	Expected Benefit
Team Selection	Use predictive analytics to support squad selection by evaluating combinations of batting, bowling, and all-rounder strengths.	More balanced team composition and improved probability of success.
Opposition Analysis	Develop opponent-specific preparation strategies using historical performance trends.	Better tactical planning against stronger opponents.
Bowling Performance	Monitor bowling discipline, particularly extra deliveries, as a key performance indicator during training.	Reduced scoring opportunities for opponents and improved defensive performance.
Match Preparation	Incorporate historical match conditions and format-specific insights into pre-match planning.	Better tactical decisions across ODI, Test, and T20 formats.
Performance Monitoring	Continuously update predictive models with recent match data to reflect changes in player form and team strategies.	Improved prediction accuracy and more relevant decision support over time.

The results of this study demonstrate that predictive analytics can complement traditional cricket expertise by providing objective, data-driven insights. Rather than replacing the experience of coaches and selectors, the developed model should be used as a decision-support tool that assists in evaluating tactical alternatives and identifying factors associated with successful match outcomes.

Although the developed model achieved strong predictive performance, several limitations should be acknowledged.

- The analysis was based solely on historical match data.
- Real-time factors such as player injuries, weather conditions, pitch reports, and player fitness were not included.
- The dataset represents historical patterns and may not fully capture future changes in team strategies or player performance.
- Match outcomes are influenced by unpredictable events that cannot always be represented using structured historical data.

Recognising these limitations provides context for interpreting the model results and identifies opportunities for future improvement.

C O N C L U S I O N

This study successfully developed and evaluated multiple machine learning models for predicting international cricket match outcomes. Exploratory analysis identified meaningful relationships within the dataset, while systematic preprocessing ensured that the data was suitable for predictive modelling.

Among the evaluated algorithms, Gradient Boosting demonstrated the strongest overall performance and was selected as the final predictive model. The findings illustrate that machine learning provides valuable decision-support capabilities that can assist the BCCI in team selection, tactical planning, and performance analysis.

Overall, the study demonstrates the potential of data-driven analytics to complement traditional cricket expertise and support more informed strategic decision-making.

7. APPENDIX

Appendix A - Data Loading & Initial Exploration

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (confusion_matrix, classification_report, accuracy_score)

from sklearn.metrics import (roc_auc_score, roc_curve)

from sklearn.tree import DecisionTreeClassifier

from sklearn import tree

from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import GridSearchCV

from sklearn.ensemble import GradientBoostingClassifier
```

```
df= pd.read_excel("Sports Data.xlsx", sheet_name= "Sports data for DSBA")

df.head() Complete raw output is provided in Appendix Spreadsheet under sheet name – Data Preview
```

```
df.shape

(2923, 23)
```

```
df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2930 entries, 0 to 2929

Data columns (total 23 columns):

 #   Column                  Non-Null Count Dtype

---  ------                  --------------  -----

 0   Game_number             2930 non-null   object

 1   Result                  2930 non-null   object

 2   Avg_team_Age            2833 non-null   float64

 3   Match_light_type        2878 non-null   object

 4   Match_format            2860 non-null   object

 5   Bowlers_in_team         2848 non-null   float64

 6   Wicket_keeper_in_team   2930 non-null   int64

 7   All_rounder_in_team     2890 non-null   float64

 8   First_selection         2871 non-null   object

 9   Opponent                2894 non-null   object

 10  Season                  2868 non-null   object

 11  Audience_number         2849 non-null   float64

 12  Offshore                2866 non-null   object

 13  Max_run_scored_1over    2902 non-null   float64

 14  Max_wicket_taken_1over  2930 non-null   int64

 15  Extra_bowls_bowled      2901 non-null   float64

 16  Min_run_given_1over     2930 non-null   int64

 17  Min_run_scored_1over    2903 non-null   float64
```

```
 18  Max_run_given_1over     2896 non-null   float64

 19  extra_bowls_opponent    2930 non-null   int64

 20  player_highest_run      2902 non-null   float64

 21  Players_scored_zero     2930 non-null   object

 22  player_highest_wicket   2930 non-null   object

dtypes: float64(9), int64(4), object(10)

memory usage: 526.6+ KB
```

```
df.describe().T – Raw output provided in Appendix Spreadsheet - sheet name Descriptive Statistics

df.describe(include='object').T - Raw output provided in Appendix Spreadsheet - sheet name
Descriptive Statistics
```

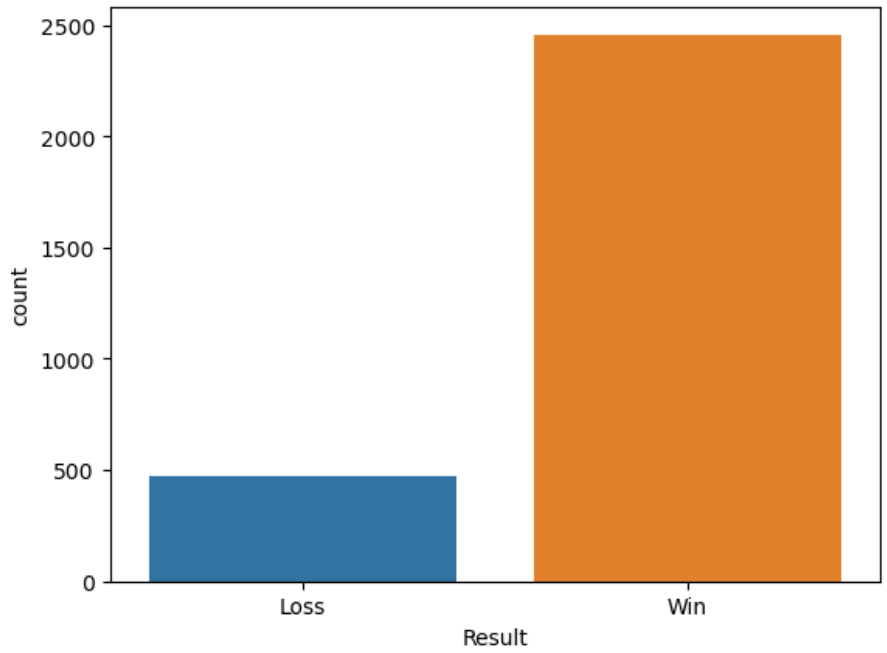
Appendix B – Exploratory Data Analysis: Univariate Analysis

Match Results

Figure_B1 Distribution of Match Results

```
sns.countplot(x='Result', data=df)

plt.show()
```



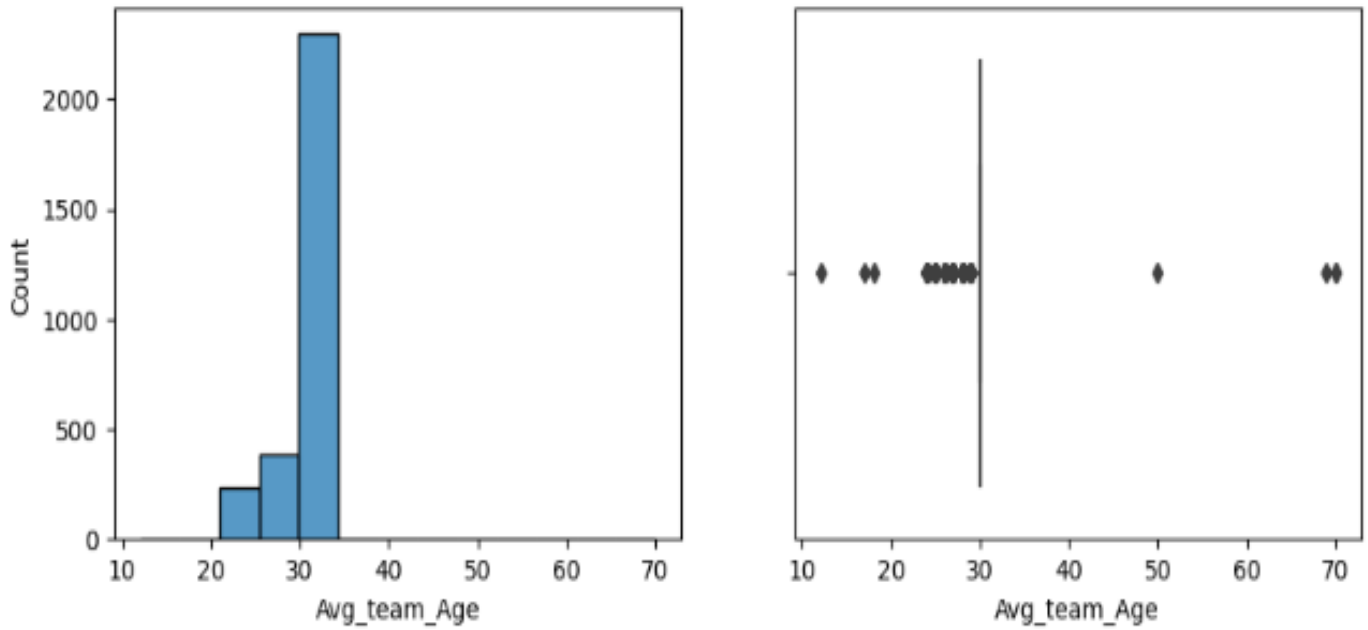
Average Team Age

Figure_B2 Distribution of Average Team Age

```
fig, axes = plt.subplots(nrows=3, ncols=2, figsize=(10,12))

sns.histplot(df['Avg_team_Age'], kde=False, ax=axes[0])

sns.boxplot(x='Avg_team_Age', data=df, ax=axes[1])
```

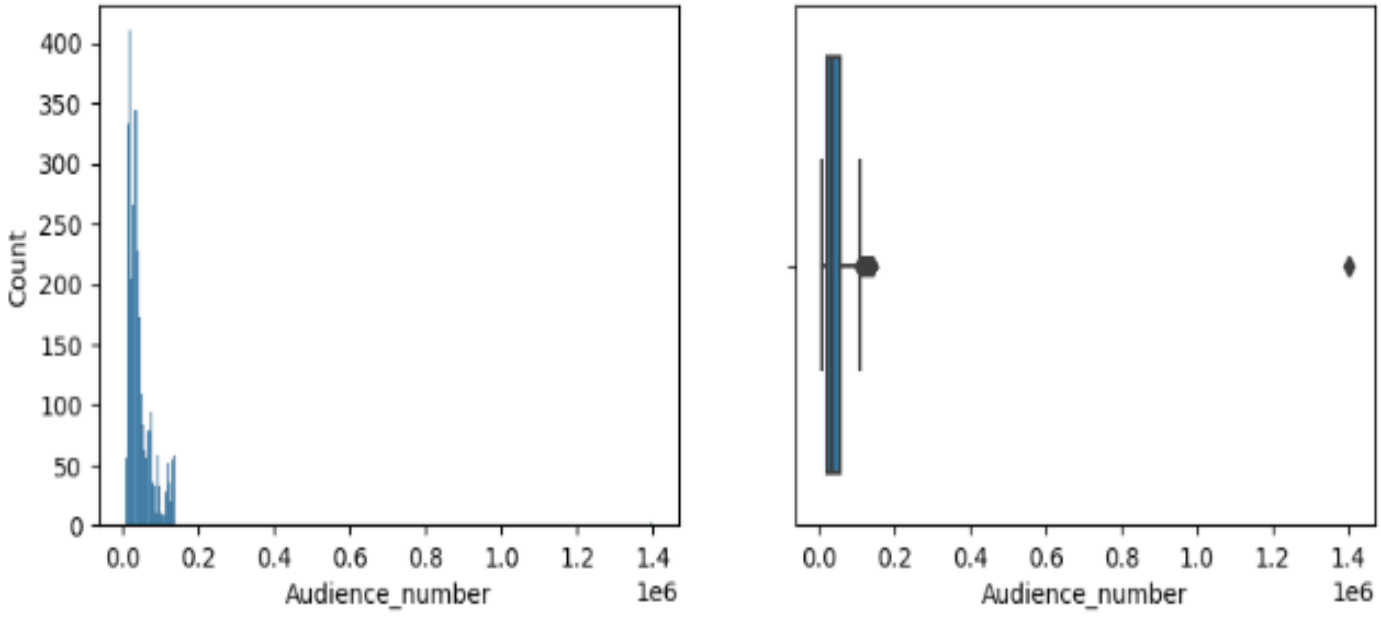


Audience Attendance

Figure_B3 Distribution of Audience Attendance

```
sns.histplot(df['Audience_number'], kde=False, ax=axes[0])

sns.boxplot(x='Audience_number', data=df, ax=axes[1])
```



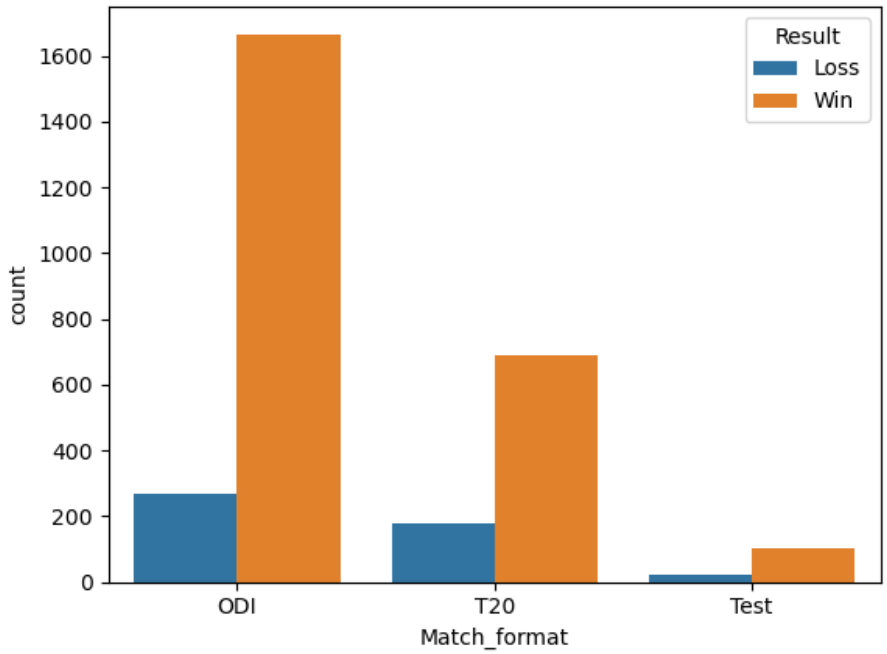
Bivariate Analysis

Match Format and Match Result

Figure_B4 Match Format vs Result

```
sns.countplot(x='Match_format', hue='Result', data=df)

plt.show;
```



Table_B4 Crosstab percentage of Match Format and Result

```
pd.crosstab(df['Match_format'], df['Result'], normalize='index') * 100
```

Match_format	Result %	
	Loss	Win
ODI	13.9	86.09
T20	20.68	79.31
Test	19.2	80.8

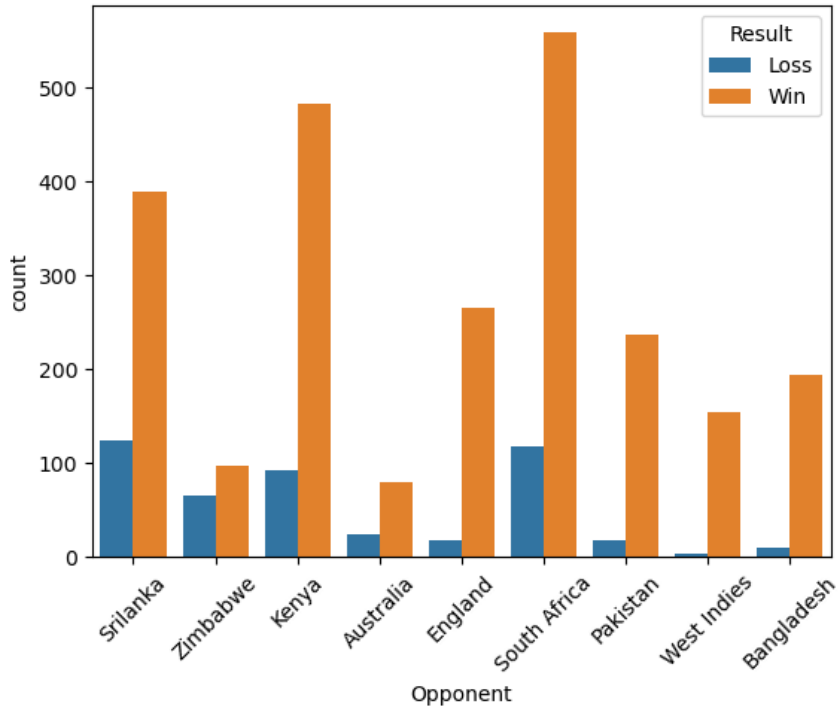
Opponent and Result

```
sns.countplot(x='Opponent', hue='Result', data=df)

plt.xticks(rotation=45)

plt.show()
```

Figure_B5 Opponent vs Result



Table_B5 Percentage distribution of Match Outcomes by Opposing Team

```
pd.crosstab(df['Opponent'], df['Result'], normalize='index') * 100
```

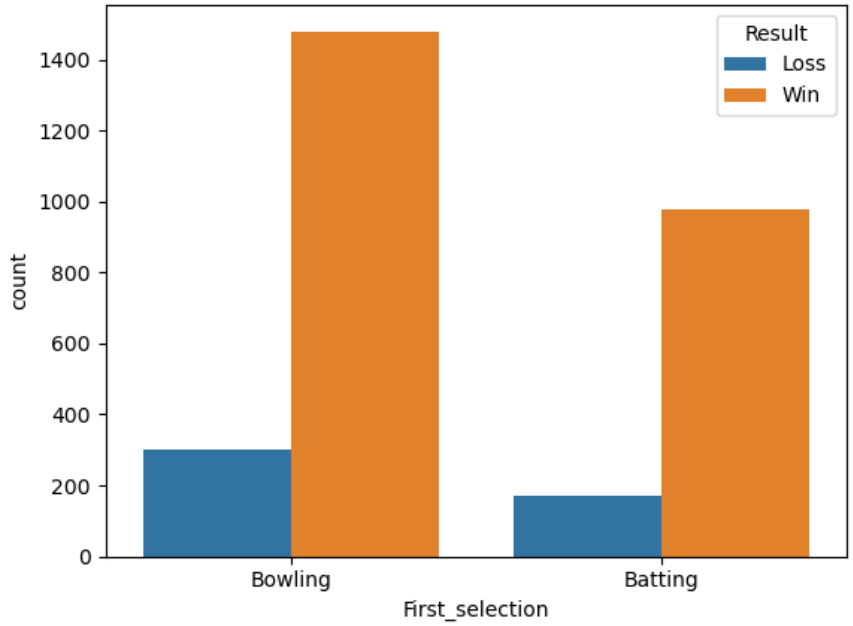
Opponent	Result %	
	Loss	Win
Australia	23.1	76.9
Bangladesh	4.9	95.1
England	6.36	93.6
Kenya	16.2	83.9
Pakistan	6.72	93.3
South Africa	17.3	82.7
Srilanka	24.2	75.8
West Indies	2.53	97.5
Zimbabwe	40.5	59.5

First Selection and Result

```
sns.countplot(x='First_selection', hue='Result', data=df)

plt.show()
```

Figure_B6 First Selection vs Result



Table_B6 Percentage Distribution of Match outcomes by Toss Decision

```
pd.crosstab(df['First_selection'], df['Result'], normalize='index') * 100
```

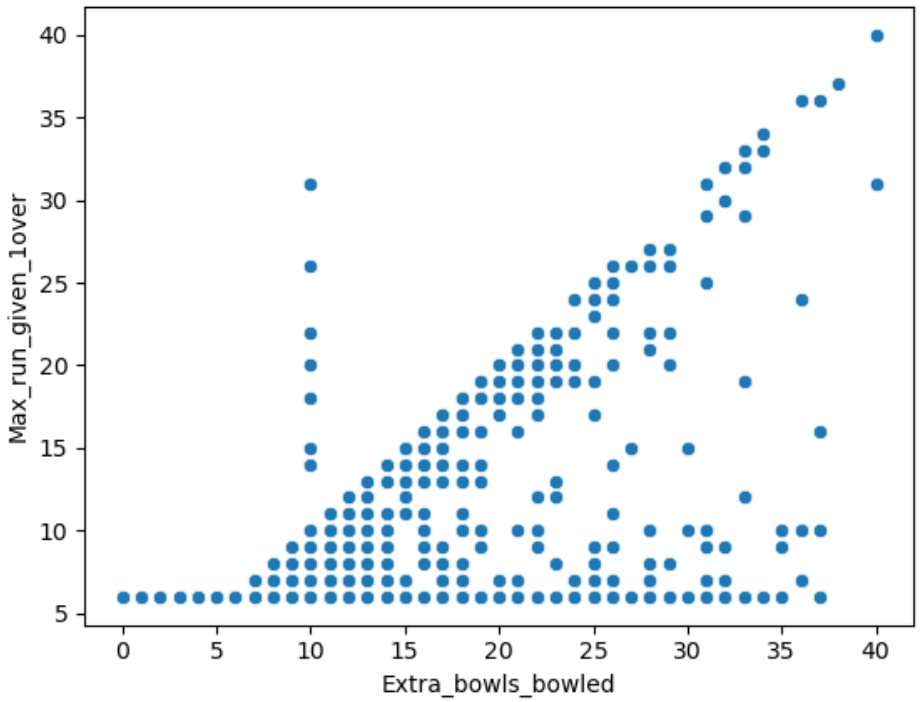
First_selection	Result %	
	Loss	Win
Batting	14.96	85.03
Bowling	16.9	83.09

Extra Deliveries and Max Runs

Figure_B7 Extra bowls bowled vs max runs given 1 over

```
sns.scatterplot(x='Extra_bowls_bowled', y='Max_run_given_1over', data=df)

plt.show()
```



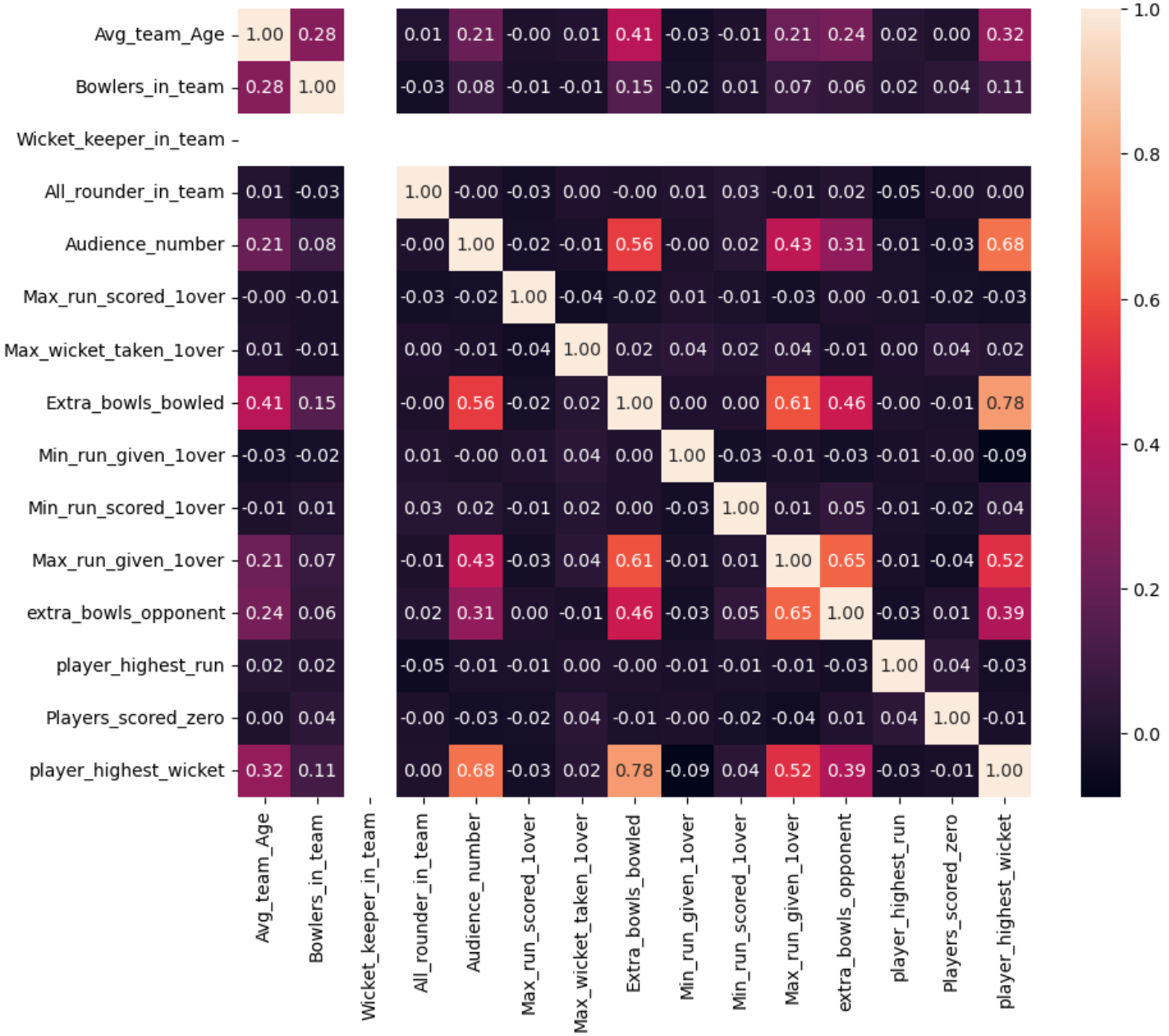
Multivariate Analysis

Correlation Heatmap

Figure_B8 Correlation Heatmap

```
plt.figure(figsize=(10,8))

sns.heatmap(df.corr(), annot=True,fmt='.2f');
```


Avg_team_Age 1.00 0.28 0.01 0.21 -0.00 0.01 0.41 -0.03 -0.01 0.21 0.24 0.02 0.00 0.32
Bowlers_in_team 0.28 1.00 -0.03 0.08 -0.01 -0.01 0.15 -0.02 0.01 0.07 0.06 0.02 0.04 0.11
Wicket_keeper_in_team
All_rounder_in_team 0.01 -0.03 1.00 -0.00 -0.03 0.00 -0.00 0.01 0.03 -0.01 0.02 -0.05 -0.00 0.00
Audience_number 0.21 0.08 -0.00 1.00 -0.02 -0.01 0.56 -0.00 0.02 0.43 0.31 -0.01 -0.03 0.68
Max_run_scored_1over -0.00 -0.01 -0.03 -0.02 1.00 -0.04 -0.02 0.01 -0.01 -0.03 0.00 -0.01 -0.02 -0.03
Max_wicket_taken_1over 0.01 -0.01 0.00 -0.01 -0.04 1.00 0.02 0.04 0.02 0.04 -0.01 0.00 0.04 0.02
Extra_bowls_bowled 0.41 0.15 -0.00 0.56 -0.02 0.02 1.00 0.00 0.00 0.61 0.46 -0.00 -0.01 0.78
Min_run_given_1over -0.03 -0.02 0.01 -0.00 0.01 0.04 0.00 1.00 -0.03 -0.01 -0.03 -0.01 -0.00 -0.09
Min_run_scored_1over -0.01 0.01 0.03 0.02 -0.01 0.02 0.00 -0.03 1.00 0.01 0.05 -0.01 -0.02 0.04
Max_run_given_1over 0.21 0.07 -0.01 0.43 -0.03 0.04 0.61 -0.01 0.01 1.00 0.65 -0.01 -0.04 0.52
extra_bowls_opponent 0.24 0.06 0.02 0.31 0.00 -0.01 0.46 -0.03 0.05 0.65 1.00 -0.03 0.01 0.39
player_highest_run 0.02 0.02 -0.05 -0.01 -0.01 0.00 -0.00 -0.01 -0.01 -0.01 -0.03 1.00 0.04 -0.03
Players_scored_zero 0.00 0.04 -0.00 -0.03 -0.02 0.04 -0.01 -0.00 -0.02 -0.04 0.01 0.04 1.00 -0.01
player_highest_wicket 0.32 0.11 0.00 0.68 -0.03 0.02 0.78 -0.09 0.04 0.52 0.39 -0.03 -0.01 1.00
Avg_team_Age
Bowlers_in_team
Wicket_keeper_in_team
All_rounder_in_team
Audience_number
Max_run_scored_1over
Max_wicket_taken_1over
Extra_bowls_bowled
Min_run_given_1over
Min_run_scored_1over
Max_run_given_1over
extra_bowls_opponent
player_highest_run
Players_scored_zero
player_highest_wicket
1.0
0.8
0.6
0.4
0.2
0.0

Appendix C. Data Cleaning & Pre-processing

C1. Initial Data inspection

```
df.isna().sum()

Game_number                0
Result                     0
Avg_team_Age              97
Match_light_type          52
Match_format              70
Bowlers_in_team           82
Wicket_keeper_in_team      0
All_rounder_in_team       40
First_selection           59
Opponent                  36
Season                    62
Audience_number           81
Offshore                  64
Max_run_scored_1over      28
Max_wicket_taken_1over     0
Extra_bowls_bowled        29
Min_run_given_1over        0
Min_run_scored_1over      27
Max_run_given_1over       34
extra_bowls_opponent       0
player_highest_run        28
```

```
Players_scored_zero        0

player_highest_wicket      0

dtype: int64
```

Appendix_C2. Missing Value Treatment

```
df_num = df.select_dtypes(['float64', 'int64'])

df_cat = df.select_dtypes(['object'])
```

```
df_num.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2930 entries, 0 to 2929

Data columns (total 15 columns):

 #   Column                  Non-Null Count  Dtype

---  ------                  --------------  -----

 0   Avg_team_Age            2833 non-null   float64

 1   Bowlers_in_team         2848 non-null   float64

 2   Wicket_keeper_in_team   2930 non-null   int64

 3   All_rounder_in_team     2890 non-null   float64

 4   Audience_number         2849 non-null   float64

 5   Max_run_scored_1over    2902 non-null   float64

 6   Max_wicket_taken_1over  2930 non-null   int64

 7   Extra_bowls_bowled      2901 non-null   float64

 8   Min_run_given_1over     2930 non-null   int64

 9   Min_run_scored_1over    2903 non-null   float64

 10  Max_run_given_1over     2896 non-null   float64
```

```
 11  extra_bowls_opponent    2930 non-null   int64
 12  player_highest_run      2902 non-null   float64
 13  Players_scored_zero     2930 non-null   int64
 14  player_highest_wicket   2930 non-null   int64
dtypes: float64(9), int64(6)
memory usage: 343.5 KB
```

```
df_cat.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2930 entries, 0 to 2929
Data columns (total 8 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Game_number       2930 non-null   object
 1   Result            2930 non-null   object
 2   Match_light_type  2878 non-null   object
 3   Match_format      2860 non-null   object
 4   First_selection   2871 non-null   object
 5   Opponent          2894 non-null   object
 6   Season            2868 non-null   object
 7   Offshore          2866 non-null   object
dtypes: object(8)
memory usage: 183.3+ KB
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(missing_values= np.nan, strategy= 'median')

imr = SimpleImputer(missing_values= np.nan, strategy= 'most_frequent')

df_num = pd.DataFrame(imputer.fit_transform(df_num), columns = df_num.columns)

df_cat = pd.DataFrame(imr.fit_transform(df_cat), columns = df_cat.columns)
```

```
df_num.isnull().sum()

Avg_team_Age              0

Bowlers_in_team           0

Wicket_keeper_in_team     0

All_rounder_in_team       0

Audience_number           0

Max_run_scored_1over      0

Max_wicket_taken_1over    0

Extra_bowls_bowled        0

Min_run_given_1over       0

Min_run_scored_1over      0

Max_run_given_1over       0

extra_bowls_opponent      0

player_highest_run        0

Players_scored_zero       0

player_highest_wicket     0

dtype: int64
```

```
df_cat.isnull().sum()

Game_number         0

Result              0

Match_light_type    0

Match_format        0

First_selection     0

Opponent            0

Season              0

Offshore            0

dtype: int64
```

```
df_new = pd.concat([df_num, df_cat], axis=1, join='inner')

df= df_new
```

Appendix_C3 Outlier Assessment and Treatment

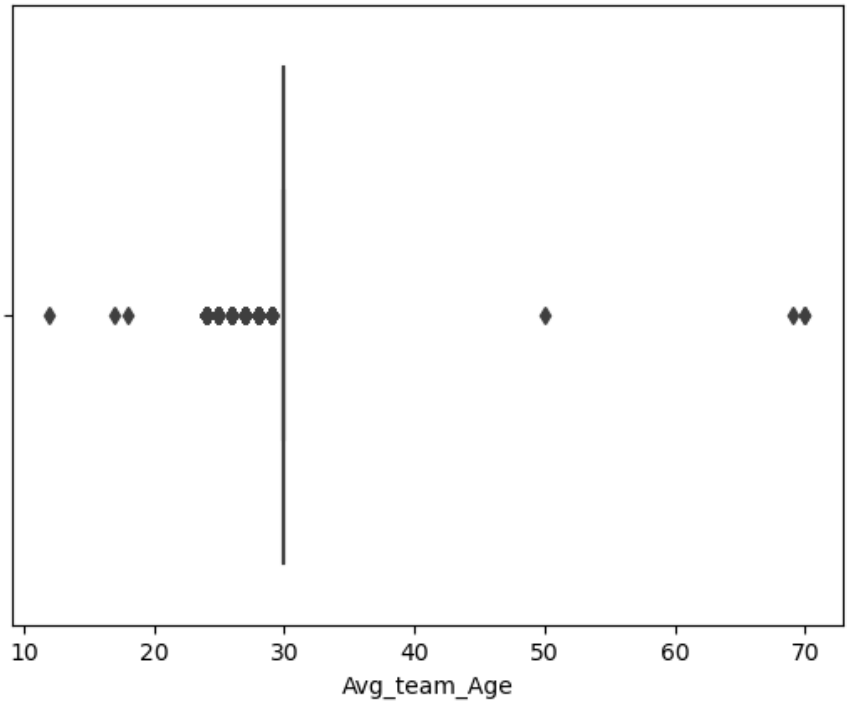
```
df.Avg_team_Age.unique()

array([18., 24., nan, 17., 12., 25., 26., 27., 28., 29., 30., 70., 69.,

       50.])
```

```
sns.boxplot(x='Avg_team_Age', data=df);
```

Figure_C3 – Average Team Age




```
df[(df['Avg_team_Age']<20) | (df['Avg_team_Age']>40)] – Raw output in Appendix Spreadsheet: Sheet name - Outlier Treatment
```

```
df=df[(df['Avg_team_Age'] >=20) & (df['Avg_team_Age'] <=40)]
```

```
df['Avg_team_Age'].value_counts()

30.0    2300

24.0     184

29.0     131

27.0      92

28.0      92

26.0      74

25.0      50

Name: Avg_team_Age, dtype: int64
```

```
df['Avg_team_Age'].describe()

count    2923.000000

mean       29.233322

std         1.722230

min        24.000000

25%        30.000000

50%        30.000000

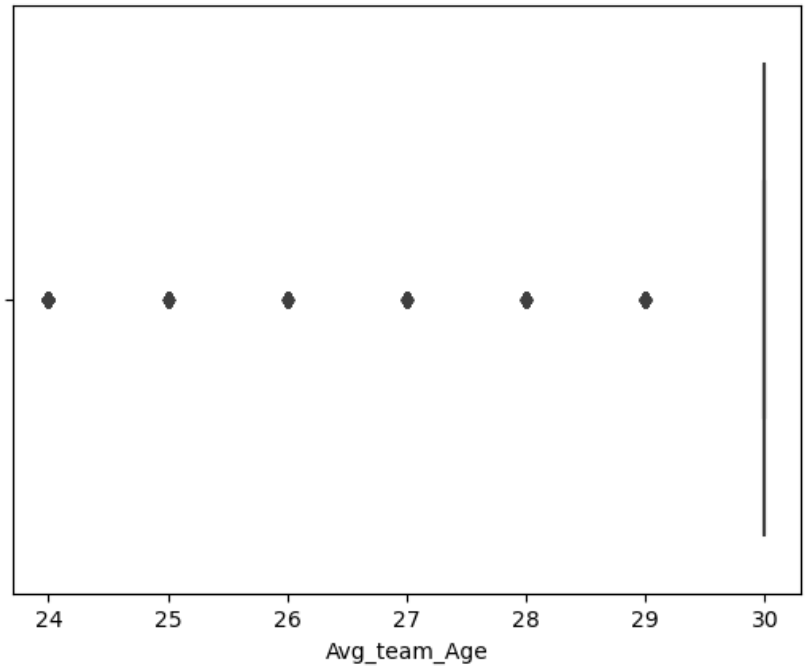
75%        30.000000

max        30.000000

Name: Avg_team_Age, dtype: float64
```

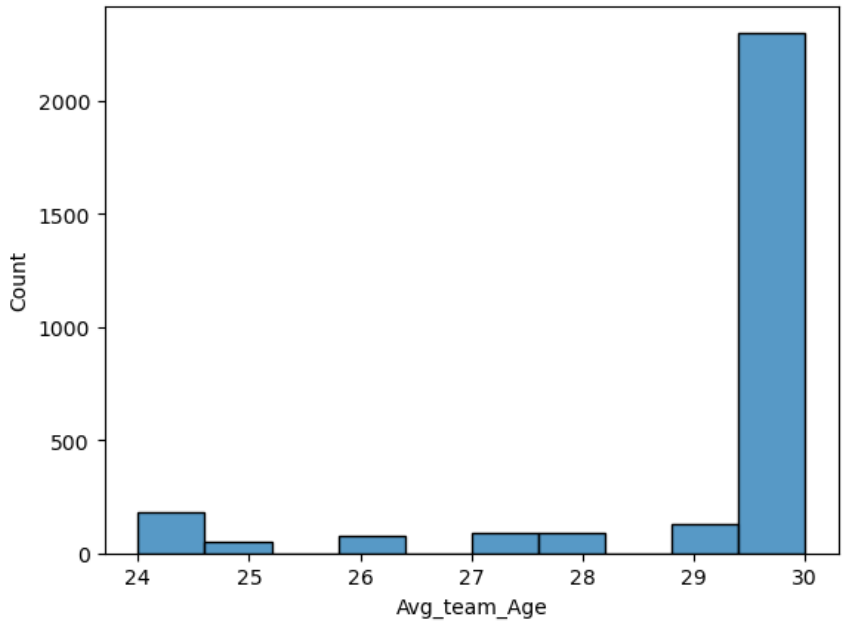
Figure_C3.2

```
sns.boxplot(x='Avg_team_Age', data=df);
```



Figure_C3.3

```
sns.histplot(df['Avg_team_Age'], bins=10);
```



Appendix_C4 Variable Transformation

Data Type Conversions

```
int_cols = ['Bowlers_in_team',
'All_rounder_in_team','Min_run_scored_1over','Max_run_scored_1over','Max_run_given_1over',
'Extra_bowls_bowled','Min_run_given_1over','player_highest_run',
'Wicket_keeper_in_team','Max_wicket_taken_1over','extra_bowls_opponent','Players_scored_zero',

'player_highest_wicket']

df[int_cols] = df[int_cols].astype('Int64')
```

```
df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2930 entries, 0 to 2929

Data columns (total 23 columns):

 #   Column                  Non-Null Count  Dtype

---  ------                  --------------  -----

 0   Avg_team_Age            2930 non-null   float64

 1   Bowlers_in_team         2930 non-null   Int64

 2   Wicket_keeper_in_team   2930 non-null   Int64

 3   All_rounder_in_team     2930 non-null   Int64

 4   Audience_number         2930 non-null   float64

 5   Max_run_scored_1over    2930 non-null   Int64
```

```
 6   Max_wicket_taken_1over  2930 non-null   Int64
 7   Extra_bowls_bowled      2930 non-null   Int64
 8   Min_run_given_1over     2930 non-null   Int64
 9   Min_run_scored_1over    2930 non-null   Int64
 10  Max_run_given_1over     2930 non-null   Int64
 11  extra_bowls_opponent    2930 non-null   Int64
 12  player_highest_run      2930 non-null   Int64
 13  Players_scored_zero     2930 non-null   Int64
 14  player_highest_wicket   2930 non-null   Int64
 15  Game_number             2930 non-null   object
 16  Result                  2930 non-null   object
 17  Match_light_type        2930 non-null   object
 18  Match_format            2930 non-null   object
 19  First_selection         2930 non-null   object
 20  Opponent                2930 non-null   object
 21  Season                  2930 non-null   object
 22  Offshore                2930 non-null   object
dtypes: Int64(13), float64(2), object(8)
memory usage: 563.8+ KB
```

Standardization for K-Means

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_model_train)
X_test_scaled = scaler.transform(X_model_test)
```

Cluster variable Creation

```
df_kmeans = df.drop(columns=['Game_number', 'Audience_number', 'Wicket_keeper_in_team'])
```

```
cat=[]

num=[]

for i in df_kmeans.columns:

    if df_kmeans[i].dtype=='object':

        cat.append(i)

    else:

        num.append(i)

print(cat)

print(num)
```

```
['Result', 'Match_light_type', 'Match_format', 'First_selection', 'Opponent', 'Season',
'Offshore']

['Avg_team_Age', 'Bowlers_in_team', 'All_rounder_in_team', 'Max_run_scored_1over',
'Max_wicket_taken_1over', 'Extra_bowls_bowled', 'Min_run_given_1over', 'Min_run_scored_1over',
'Max_run_given_1over', 'extra_bowls_opponent', 'player_highest_run', 'Players_scored_zero',
'player_highest_wicket']
```

```
features = [

    'Extra_bowls_bowled',

    'player_highest_run',

    'player_highest_wicket',

    'Max_run_given_1over',

    'Min_run_given_1over',

    'Min_run_scored_1over'

]
```

```
X= df_kmeans[features]

scaler = StandardScaler()

X_scaled= scaler.fit_transform(X)

wss =[]

for i in range(1,11):

    KM =KMeans(n_clusters=i)

    KM.fit(X_scaled)

    wss.append(KM.inertia_)
```

```
wss

[17538.000000000004,

 12939.579167155169,

 11107.694856236485,

 9900.82076407322,

 8768.43273718186,

 7830.747044375212,

 7331.202134048743,

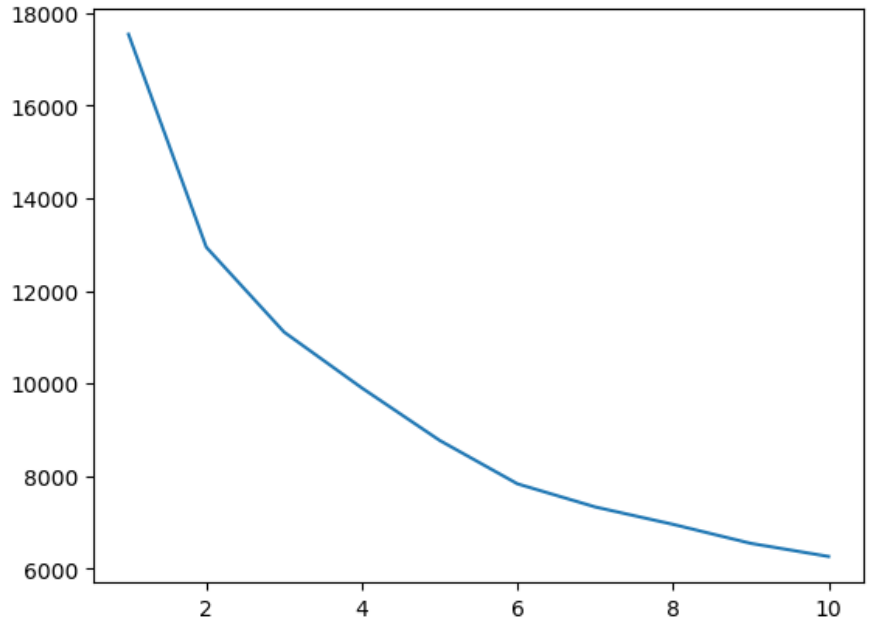
 6955.928620720966,

 6545.186376238554,

 6262.3632397906285]
```

Figure_C4.1

```
plt.plot(range(1,11), wss)
```




```
k_means =KMeans(n_clusters =3)

k_means.fit(X_scaled)

labels = k_means.labels_
```

```
df_profile = df_kmeans.copy()

df_profile['Cluster']= labels

df_profile.head(10) – Raw output in Appendix Spreadsheet – Sheet name: Kmeans
```

Variable removed for Classification

```
X_model = df.drop(columns=['Result', 'Game_number', 'Audience_number', 'Wicket_keeper_in_team'])

y = df['Result']

X_model = pd.get_dummies(X_model, drop_first=True)
```

```
X_model.head()- Raw output in Appendix Spreadsheet – sheet name: Variable Removal
```

Variables removed for K-Means

```
df_kmeans = df.drop(columns=['Game_number', 'Audience_number', 'Wicket_keeper_in_team'])
```

Appendix D – Model Building

Model Selection Strategy

D1. Logistic Regression

```
log_model = LogisticRegression(class_weight='balanced', max_iter=1000)

log_model.fit(X_train_scaled, train_labels)
```

D1.2 Decision Tree

```
dt_model = DecisionTreeClassifier(criterion = 'gini', class_weight = 'balanced', random_state=1)

dt_model.fit(X_model_train, train_labels)
```

D1.3 Random Forest

```
rf_model = RandomForestClassifier(n_estimators = 200, class_weight = 'balanced', random_state = 1,
oob_score=True)

rf_model = rf_model.fit(X_model_train, train_labels)
```

D1.4 Gradient Boosting

```
gb_model = GradientBoostingClassifier(random_state=1)

gb_model.fit(X_model_train, train_labels)
```

D2 Model Development

D2.1 Train-Test Split

```
X_model_train, X_model_test, train_labels, test_labels=train_test_split(X_model, y, test_size=.30,
random_state=1, stratify=y)
```

D3 Improving Model Performance

D3.1 Decision Tree Optimisation

```
clf_dt_model = DecisionTreeClassifier(criterion = 'gini',

                                      class_weight = 'balanced',

                                      random_state=1,

                                      max_depth = 16,

                                      min_samples_leaf=1,

                                      min_samples_split=3)
```

D3.2 Random Forest Hyperparameter Tuning

```
grid_search = GridSearchCV(estimator = rfcl, param_grid = param_grid, cv = 3)

grid_search.fit(X_model_train, train_labels)
```

```
grid_search.best_params_

{'class_weight': 'balanced',

 'max_depth': 15,

 'max_features': 6,

 'min_samples_leaf': 5,

 'min_samples_split': 10,

 'n_estimators': 200}
```

D3.3 Gradient Boosting Hyperparameter Tuning

```
grid_search_gb = GridSearchCV(estimator = gbcl, param_grid = param_grid_gb, cv = 3)

grid_search_gb.fit(X_model_train, train_labels)
```

```
grid_search_gb.best_params_

{'learning_rate': 0.2,

 'max_depth': 15,

 'max_features': 5,

 'min_samples_leaf': 5,

 'min_samples_split': 20,

 'n_estimators': 200}
```

Appendix E – Model Validation

E1 Logistic Regression

```
print("Train Accuracy:", log_model.score(X_train_scaled, train_labels))

print("Test Accuracy:", log_model.score(X_test_scaled, test_labels))

print(confusion_matrix(test_labels, ytest_pred_lr))

print(classification_report(test_labels, ytest_pred_lr))
```

```
Train Accuracy: 0.7487781036168133

Test Accuracy: 0.7377423033067275

[[103  38]

 [192 544]]

              precision    recall  f1-score   support


        Loss       0.35      0.73      0.47       141

         Win       0.93      0.74      0.83       736


    accuracy                           0.74       877

   macro avg       0.64      0.73      0.65       877

weighted avg       0.84      0.74      0.77       877
```

```
probs_lr_test = log_model.predict_proba(X_test_scaled)[:, 1]

auc_lr_test = roc_auc_score(test_labels, probs_lr_test)
```

```
auc_lr_test

0.8257304193647856
```

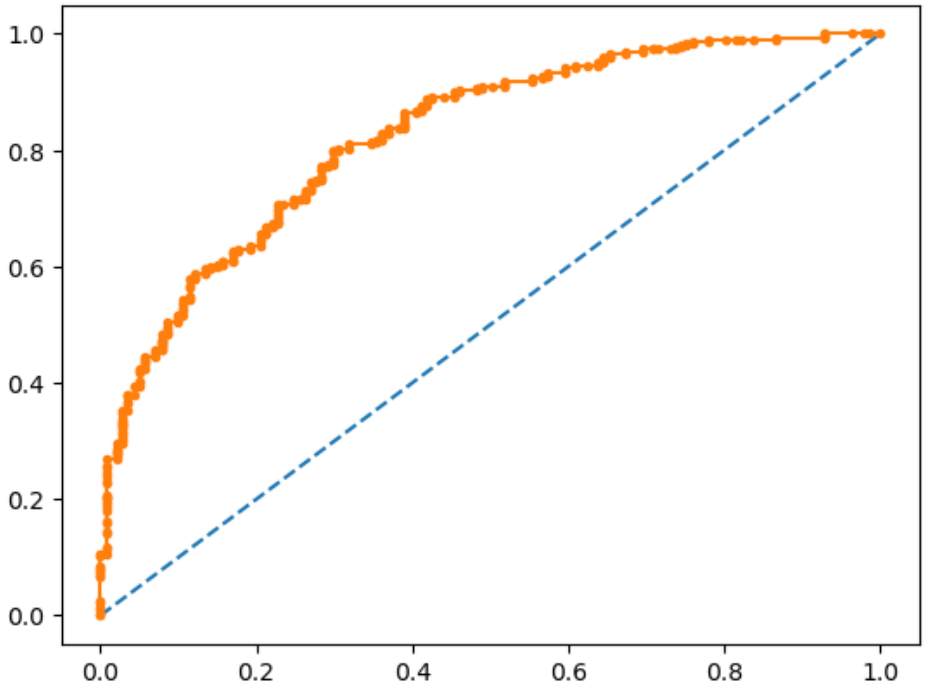
```
fpr, tpr, thresholds = roc_curve(test_labels, probs_lr_test, pos_label='Win')

plt.plot([0,1], [0,1], linestyle='--')

plt.plot(fpr, tpr, marker='.')

plt.show()
```

Figure_E1.1



E2 Decision Tree Validation

```
print(confusion_matrix(test_labels, ytest_pred_pruned))

print(classification_report(test_labels, ytest_pred_pruned))

print("Train Accuracy:",clf_dt_model.score(X_model_train, train_labels))

print("Test Accuracy:",clf_dt_model.score(X_model_test, test_labels))

[[111  30]

 [ 35 701]]

              precision    recall  f1-score   support


        Loss       0.76      0.79      0.77       141

         Win       0.96      0.95      0.96       736


    accuracy                           0.93       877

   macro avg       0.86      0.87      0.86       877

weighted avg       0.93      0.93      0.93       877


Train Accuracy: 0.9975562072336266

Test Accuracy: 0.9258836944127709
```

```
probs = clf_dt_model.predict_proba(X_model_test)

probs = probs[:, 1]

auc_test = roc_auc_score(test_labels, probs)
```

```
auc_test

0.8714731729879741
```

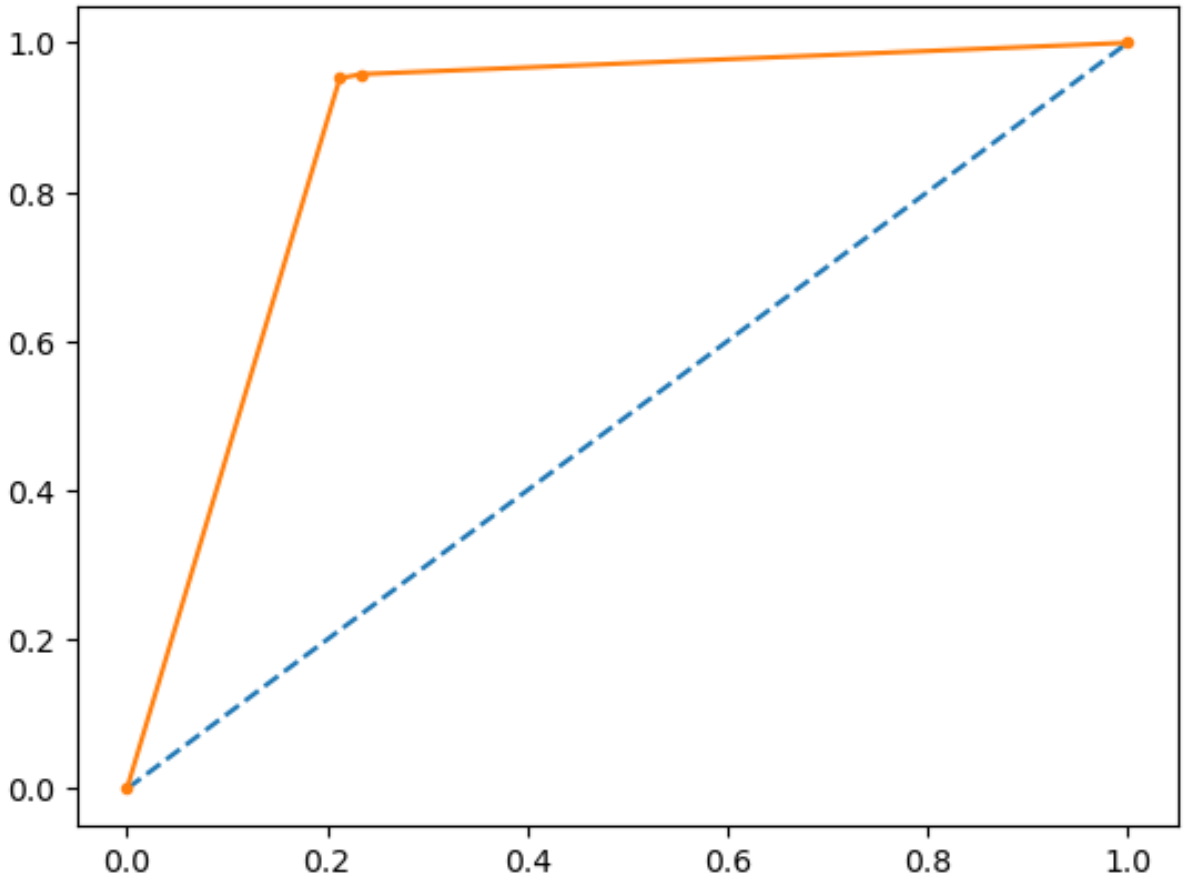
```
fpr, tpr, thresholds = roc_curve(test_labels, probs, pos_label='Win')

plt.plot([0,1], [0,1], linestyle='--')

plt.plot(fpr, tpr, marker='.')

plt.show()
```

Figure E2.1



E3 Random Forest Validation

```
print(confusion_matrix(test_labels, ytest_predict_rf))

print(classification_report(test_labels, ytest_predict_rf))

print("Train Accuracy:",best_grid.score(X_model_train, train_labels))

print("Test Accuracy:",best_grid.score(X_model_test, test_labels))

[[ 96  45]

 [ 41 695]]

              precision    recall  f1-score   support


        Loss       0.70      0.68      0.69       141

         Win       0.94      0.94      0.94       736


    accuracy                           0.90       877

   macro avg       0.82      0.81      0.82       877

weighted avg       0.90      0.90      0.90       877


Train Accuracy: 0.9648093841642229

Test Accuracy: 0.9019384264538198
```

```
probs_rf = probs_rf[:,1]

auc_rf_test = roc_auc_score(test_labels, probs_rf)
```

```
auc_rf_test

0.9323350292938637
```

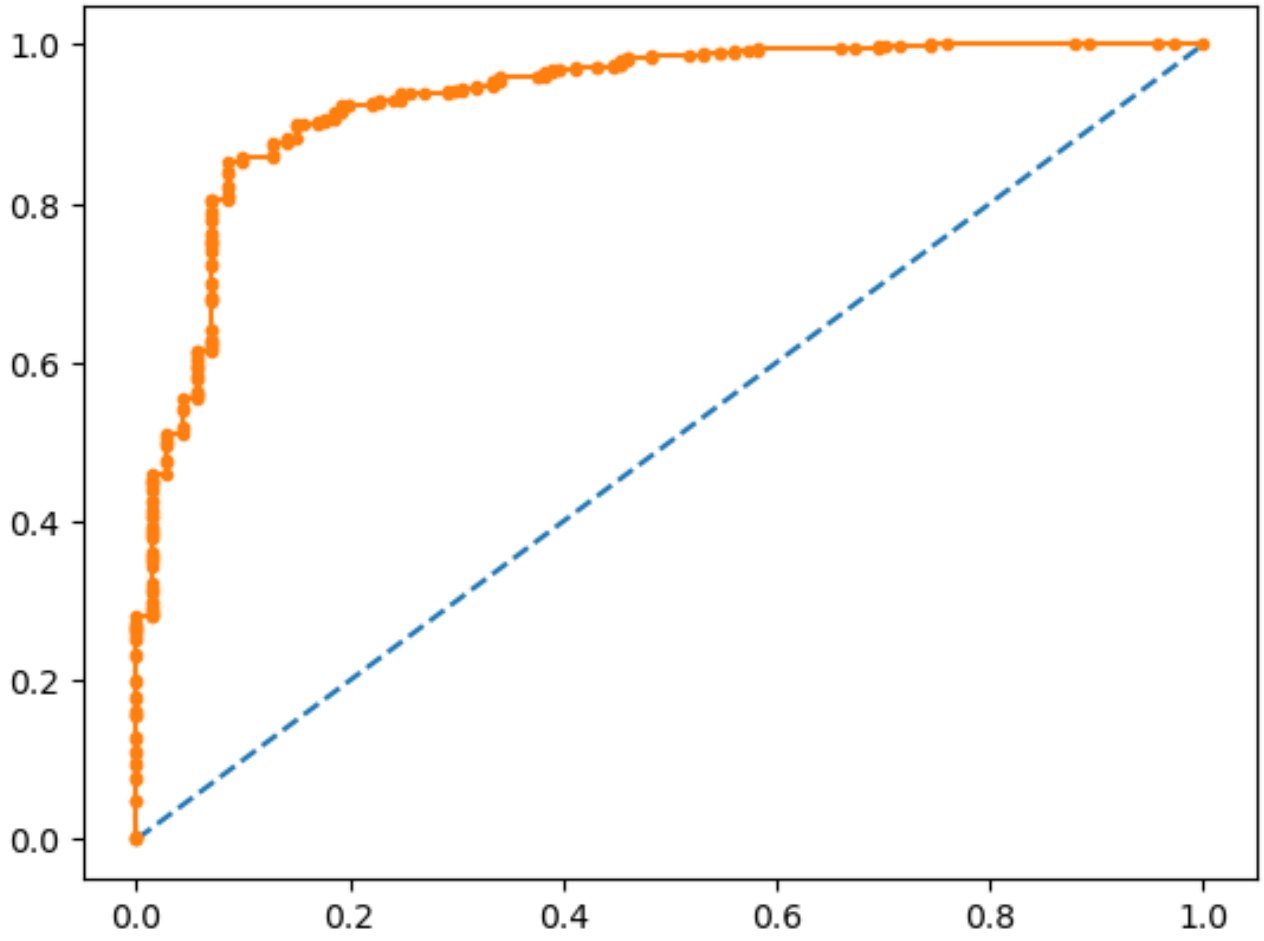
```
fpr, tpr, thresholds = roc_curve(test_labels, probs_rf, pos_label='Win')

plt.plot([0,1], [0,1], linestyle='--')

plt.plot(fpr, tpr, marker='.')

plt.show()
```

Figure E3.1



E4 Gradient Boosting Validation

```
print(confusion_matrix(test_labels, ytest_predict_gb1))

print(classification_report(test_labels, ytest_predict_gb1))

print("Train Accuracy:",best_grid_gb.score(X_model_train, train_labels))

print("Test Accuracy:",best_grid_gb.score(X_model_test, test_labels))

[[111  30]

 [  4 732]]

              precision    recall  f1-score   support


        Loss       0.97      0.79      0.87       141

         Win       0.96      0.99      0.98       736


    accuracy                           0.96       877

   macro avg       0.96      0.89      0.92       877

weighted avg       0.96      0.96      0.96       877


Train Accuracy: 1.0

Test Accuracy: 0.9612314709236032
```

```
probs_gb = probs_gb[:,1]

auc_gb_test = roc_auc_score(test_labels, probs_gb)
```

```
auc_gb_test

0.9597594819611471
```

```
fpr, tpr, thresholds = roc_curve(test_labels, probs_gb, pos_label='Win')

plt.plot([0,1], [0,1], linestyle='--')

plt.plot(fpr, tpr, marker='.')

plt.show()
```

ROC Curve

Figure E4.1

